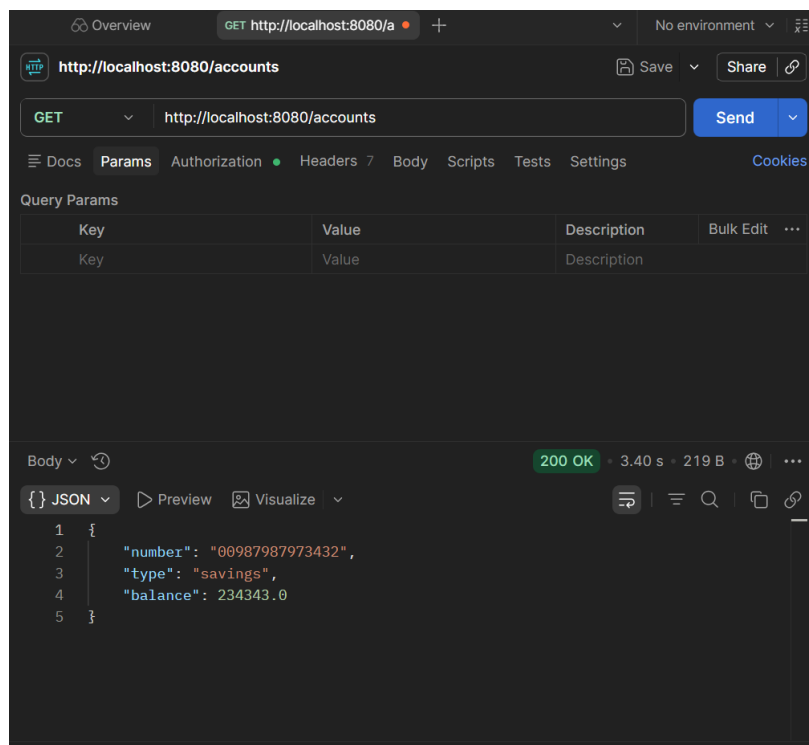


Exercise: Creating Microservices for Account and Loan

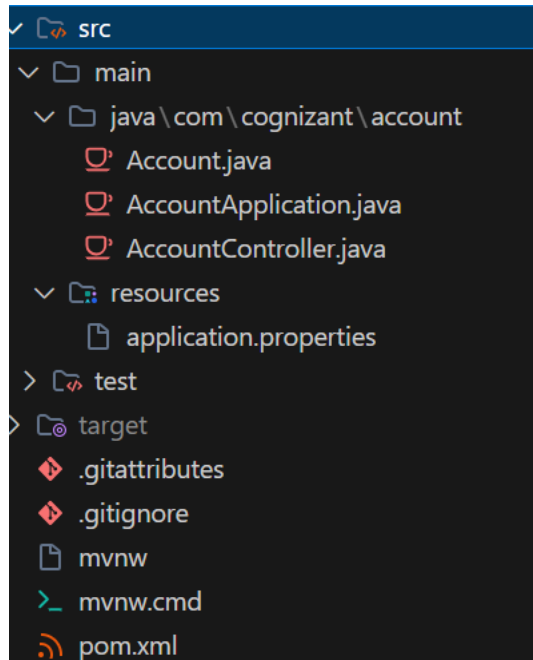
In this exercise, two independent Spring Boot microservices were developed for a banking application: **Account Service** and **Loan Service**. Each microservice was created as a separate Maven project using Spring Initializr with the required dependencies such as **Spring Web** and **Spring Boot DevTools**. The Account Service provides account details based on an account number through the endpoint `/accounts/{number}`, while the Loan Service provides loan details through the endpoint `/loans/{number}`. Instead of implementing the business logic directly in the controller, a proper layered architecture was followed by separating the **Controller** and **Service** classes, making the application more modular and maintainable. Since both services run independently, the Loan Service was configured to use **port 8081**, while the Account Service runs on the default **port 8080**, avoiding port conflicts. Finally, both microservices were executed and tested successfully using a web browser or Postman, demonstrating how microservices can be independently developed, deployed, and managed without affecting one another. This exercise illustrates the fundamental concept of microservice architecture by dividing a banking application into smaller, loosely coupled, and independently scalable services.

Results:

1. Accounts in post man



2. Accounts folder:



3. Account.java

```
import com.cognizant.account.model.Account;
import com.cognizant.account.service.AccountService;

@RestController
@RequestMapping("/accounts")
public class AccountController {

    @Autowired
    private AccountService accountService;

    @GetMapping("/{number}")
    public Account getAccount(@PathVariable String number) {
        return accountService.getAccount(number);
    }
}
```

4. Loan postman:

Overview GET http://localhost:8081/loans No environment

ht Overview Post:8081/loans Save Share

GET http://localhost:8081/loans Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK 354 ms 238 B ...

{} JSON Preview Visualize

```
1 {
2   "number": "H00987987972342",
3   "type": "car",
4   "loan": 400000.0,
5   "emi": 3258.0,
6   "tenure": 18
7 }
```

5. Loan.java

```
import com.cognizant.loan.model.Loan;
import com.cognizant.loan.service.LoanService;

@RestController
@RequestMapping("/loans")
public class LoanController {

    @Autowired
    private LoanService loanService;

    @GetMapping("/{number}")
    public Loan getLoan(@PathVariable String number) {
        return loanService.getLoan(number);
    }
}
```

6. Loan Foalder:

